Trabalho 1 - Benchmarking de Criptografia

Ana Alves - up202407579
Lara Queiroga - up202406713

1 de abril de 2026

Introdução

Neste projeto, comparamos o desempenho de três algoritmos criptográficos amplamente utilizados: AES (Advanced Encryption Standard), RSA (Rivest-Shamir-Adleman) e SHA-256 (Secure Hash Algorithm). O objetivo é analisar quanto tempo cada algoritmo demora a processar ficheiros de diferentes tamanhos e avaliar a sua eficiência em diferentes cenários.

Com este estudo, pretende-se compreender melhor a eficiência dos diferentes métodos criptográficos e a sua aplicabilidade em diversos contextos práticos.

1 Configuração Experimental

Sistema Operativo: macOS Sonoma 14.x
Máquina: MacBook Air (Model Identifier: Mac14,2)
CPU: Apple M2 (8 cores: 4 de performance e 4 de eficiência)
Memória: 8 GB RAM
Python: Python 3.9.6
Bibliotecas: `cryptography`, `timeit`, `matplotlib`, `os`, `statistics`, `hashlib`

Todos os testes foram executados localmente, garantindo um ambiente controlado.

Para assegurar significância estatística, cada operação foi repetida múltiplas vezes com o módulo `timeit`, sendo posteriormente calculados o tempo médio e o desvio padrão.

2 Alínea B: Cifragem e Decifragem com AES

B.1 - Desempenho base (Tamanhos Crescentes)

Nesta secção, apresentamos a implementação das funções de cifragem e decifragem utilizando o algoritmo AES (Advanced Encryption Standard) no modo CTR (Counter) com uma chave de 256 bits. No sequinte código mostramos a maneira que medimos os tempos de execução de cada ficheiro. Para os outros métodos mantivemos a mesma implementação então não vamos colocar no relatório esta parte de novo por questões de poupar espaço. Mas pode-se encontrar o código completo no zip nos ficheiros `rsa2048.py` e `sha256.py`.

Implementação

Listing 1: Cifragem

```python

# tamanhos dos ficheiros
sizes = [8, 64, 512, 4096, 32768, 262144, 2097152]

# chave AES e nonce fixo
key = os.urandom(32)
nonce = os.urandom(16)

# fun  es de encripta  o e decripta  o
def aes_encrypt(data):
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
    encryptor = cipher.encryptor()
    return encryptor.update(data) + encryptor.finalize()

def aes_decrypt(ciphertext):
    cipher = Cipher(algorithms.AES(key), modes.CTR(nonce))
    decryptor = cipher.decryptor()
    return decryptor.update(ciphertext) + decryptor.finalize()

# listas para guardar os tempos m dios
aes_enc = []
aes_dec = []

aes_encstd = []
aes_decstd = []

# listas para guardar os tempos de encripta  o e decripta  o do ficheiro maior
enc_2097152_runs = []
dec_2097152_runs = []

# n mero de repeti  es
repeats = 30

for s in sizes:
    with open(f"text_files/ficheiro_{s}.txt", "rb") as f:
        data = f.read()

    ciphertext = aes_encrypt(data)

    # timeit.repeat para medir m ltiplas vezes
    enc_times = timeit.repeat(lambda: aes_encrypt(data), repeat=repeats, number=1)
    dec_times = timeit.repeat(lambda: aes_decrypt(ciphertext), repeat=repeats, number=1)

    # converter para microsegundos
    enc_times_us = [t * 1e6 for t in enc_times]
```

```
    dec_times_us = [t * 1e6 for t in dec_times]

    # parte para guardar os tempos de execu  o
    if s == 2097152:
        enc_2097152_runs = enc_times_us[:10]
        dec_2097152_runs = dec_times_us[:10]

    # guarda nas listas para o gr fico a m dia e o desvio padr o
    aes_enc.append(statistics.mean(enc_times_us))
    aes_encstd.append(statistics.stdev(enc_times_us))

    aes_dec.append(statistics.mean(dec_times_us))
    aes_decstd.append(statistics.stdev(dec_times_us))
```

Resultados e Análise

O eixo das abcissas encontra-se numa escala logarítmica para facilitar a visualização de ficheiros com grandes discrepâncias de tamanho.

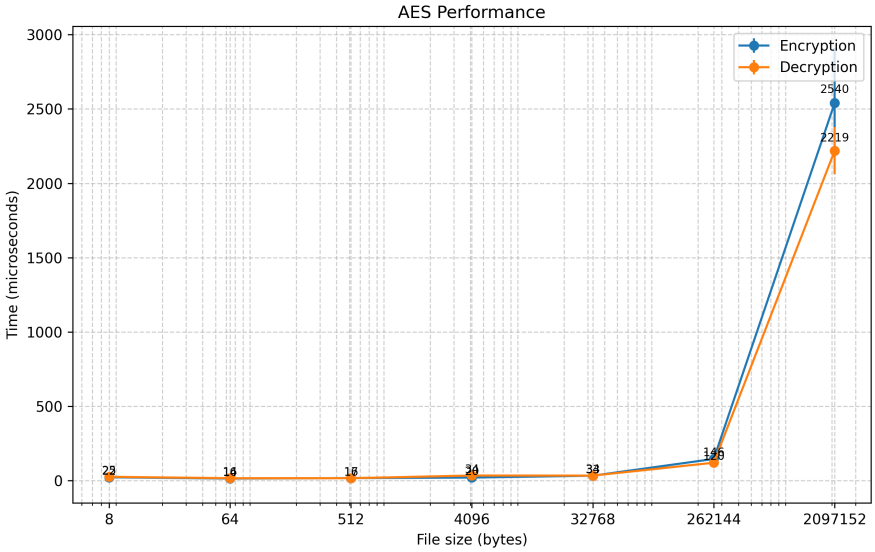

Figura 1: Desempenho da cifragem e decifragem AES (256 bits) em modo CTR.

Ao analisar o gráfico gerado, destacam-se três observações fundamentais:

- **Impacto do *Overhead* Inicial:** Para ficheiros pequenos (8 a 32768 bytes), o tempo mantém-se quase constante, sendo dominado pelo overhead das funções e pela inicialização da cifra (chave e nonce), e não pelo processamento dos dados.
- **Escalabilidade Linear:** À medida que o tamanho do ficheiro aumenta, o tempo de processamento cresce proporcionalmente, refletindo o comportamento linear do AES em modo CTR, que opera como uma cifra de fluxo.
- **Simetria Operacional:** As curvas de cifragem e decifragem apresentandam tempos muito idênticos. Achamos que seja o esperado, visto que usando AES em modo CTR a encriptação e decriptação executam a mesma operação XOR.

B.2 - Executar um algoritmo sobre o mesmo ficheiro várias vezes

Para dar resposta à segunda parte da alínea B, avaliámos se os resultados sofrem alterações significativas quando executamos um algoritmo fixo repetidamente sobre o mesmo ficheiro, em oposição à execução sobre múltiplos ficheiros distintos de tamanho fixo. Utilizamos ficheiros com 2097152 bytes.

Implementação

Listing 2: Comparação de desempenho: Mesmo Ficheiro vs Ficheiros Distintos

```
for i in range(1, 11):
    data = os.urandom(2097152)

    ciphertext = aes_encrypt(data)

    enc_time = timeit.timeit(lambda: aes_encrypt(data), number=1) *
        1e6
    dec_time = timeit.timeit(lambda: aes_decrypt(ciphertext),
       number=1) * 1e6

    enc_diff_files_runs.append(enc_time)
    dec_diff_files_runs.append(dec_time)
```

Resultados e Análise

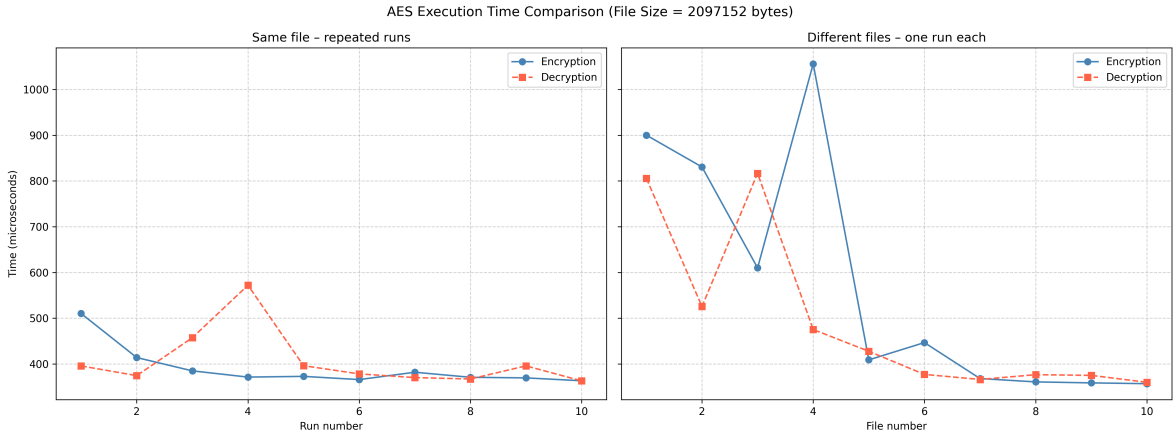

Figura 2: Comparação de desempenho: cifrar e decifrar repetidamente o mesmo ficheiro versus cifrar ficheiros distintos.

O gráfico acima corresponde a uma das vezes que realizamos a experiência, mas é importante notar que os tempos de execução variam significativamente a cada corrida do programa. Esta instabilidade pensamos que aconteça devido principalmente a dois fatores:

- **Gestão Dinâmica de Memória e *Cache*:** O sistema operativo gere a memória de forma não determinista, pelo que os dados podem ou não estar em *Cache* entre execuções. Dependendo do estado da *Cache* no momento da medição, o tempo de acesso aos dados varia consideravelmente.

- **Interferência do Sistema Operativo:** O gestor de tarefas do sistema operativo pode interromper o processo durante a medição para executar outras tarefas em segundo plano, introduzindo latência imprevisível que se reflete diretamente nos tempos registados.

3 Alínea C: Cifragem e Decifragem baseadas em RSA

Nesta secção, implementamos um esquema de cifra híbrido baseado no algoritmo assimétrico RSA (com chaves de 2048 bits) combinado com a função de *hash* SHA-256 (método sugerido pelo professor). Nós inicialmente optámos por ter um r fixo, mas depois testámos os tempos de execução com r fixo ou r's diferentes para os diferentes ficheiros e chegámos à conclusão que o tempo de execução não variava consideravelmente, então optámos por ter um novo r para cada ficheiro.

Também temos no fim do código uma pequena parte que verifica se a decriptação foi executada corretamente, isto é, se o ficheiro original é igual ao decriptado.

Implementação

Listing 3: Cifragem e decifragem utilizando esquema baseado em RSA e SHA-256

```

# chaves RSA
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
public_key = private_key.public_key()

MOD_BYTES = (public_key.public_numbers().n.bit_length() + 7) // 8

# fun  es RSA
def rsa_encrypt(r):
    r_int = bytes_to_int(r)
    c_int = pow(r_int, public_key.public_numbers().e, public_key.
       public_numbers().n)
    return int_to_bytes(c_int, MOD_BYTES)

def rsa_decrypt(c):
    c_int = bytes_to_int(c)
    r_int = pow(c_int, private_key.private_numbers().d, private_key
       .private_numbers().public_numbers.n)
    return int_to_bytes(r_int, 32)

# encripta  o e decripta  o
def encrypt_file(filename, r):
    with open(filename, "rb") as f:
        m = f.read()

    block_size = 32
    encrypted_blocks = []
```

```python

    # RSA(r)
    rsa_r = rsa_encrypt(r)

    for i in range((len(m) + block_size - 1) // block_size):
        block = m[i*block_size:(i+1)*block_size]

        hasher = hashlib.sha256()
        hasher.update(i.to_bytes(4, 'big') + r)
        hash_block = hasher.digest()

        cipher_block = bytes(a ^ b for a, b in zip(block, hash_block[:len(block)]))
        encrypted_blocks.append(cipher_block)

    return rsa_r, encrypted_blocks

def decrypt_file(rsa_r, encrypted_blocks):
    # recuperar r com a chave privada
    r = rsa_decrypt(rsa_r)

    decrypted = bytearray()

    for i, cipher_block in enumerate(encrypted_blocks):
        hasher = hashlib.sha256()
        hasher.update(i.to_bytes(4, 'big') + r)
        hash_block = hasher.digest()

        plain_block = bytes(a ^ b for a, b in zip(cipher_block, hash_block[:len(cipher_block)]))
        decrypted.extend(plain_block)

    return decrypted
```

Resultados, Análise e Comparação

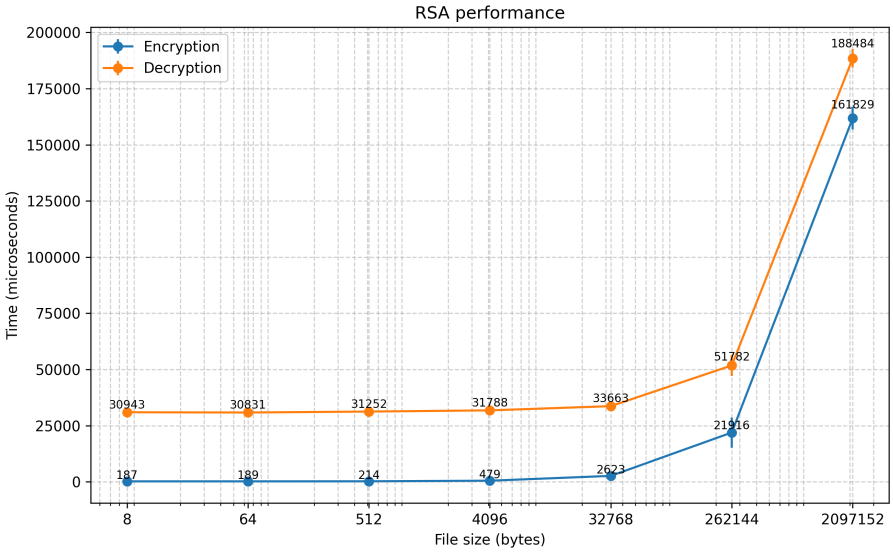

Figura 3: Desempenho da cifragem e decifragem baseadas em RSA (2048 bits).

Ao analisar o gráfico, conseguimos extrair observações cruciais sobre o comportamento deste esquema híbrido, com destaque para a comparação direta entre o processo de cifragem e o de decifragem:

- **Custo Fixo e Dominância do RSA:** Para ficheiros pequenos (até 4096 bytes), o tempo é quase constante. SHA-256 e operações XOR são rápidas, pelo que o tempo total é dominado pela exponenciação modular do RSA.
- **Assimetria de Desempenho:** A cifragem com a chave pública e é rápida, enquanto a decifragem com a chave privada d é muito mais lenta devido ao tamanho e complexidade do cálculo, gerando elevado *overhead*.
- **Crescimento Linear para Ficheiros Grandes:** A partir de 32768 bytes, ambas as curvas crescem linearmente, pois SHA-256 e XOR escalam com os dados, somando-se ao overhead inicial do RSA. Após a fase assimétrica, o processamento simétrico tem custo similar em ambos os sentidos.

4 Alínea D: Geração de Digests com SHA-256

Nesta secção, avaliamos o desempenho da função de *hash* criptográfica SHA-256.

Implementação

Listing 4: Geração de *hashes* SHA-256 e medição de desempenho

```
def sha256_hash(data):
    return hashlib.sha256(data).digest()
```

Resultados e Análise

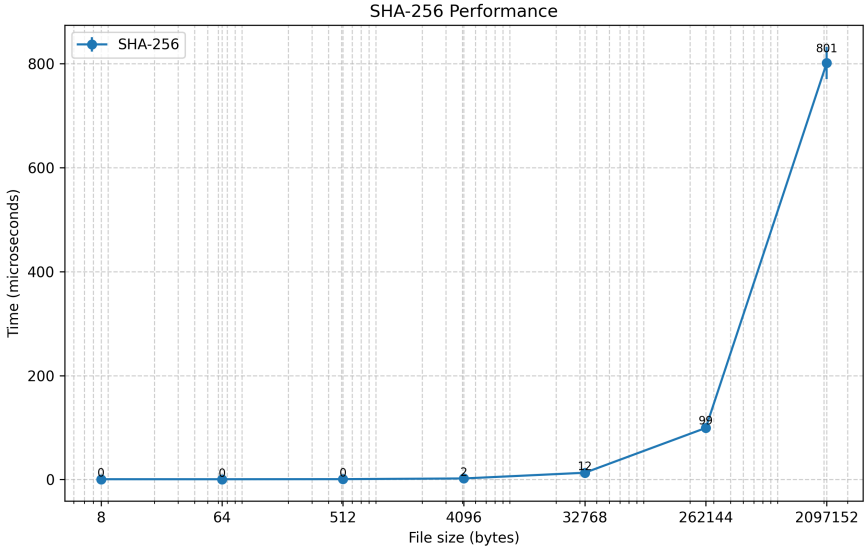

Figura 4: Desempenho da geração de digests com a função SHA-256.

Ao analisar a curva de desempenho do SHA-256, evidenciam-se dois aspetos fundamentais relativos à sua eficiência:

- **Eficiência Extrema e Baixo *Overhead*:** Para ficheiros pequenos (8 a 4096 bytes), o tempo de execução é praticamente nulo (2 µs aos 4096 bytes). Ao contrário das cifras a função de hash não requer setup complexo, gestão de chaves ou operações pesadas, sendo uma operação direta e altamente otimizada que processa quase instantâneamente os blocos iniciais.

- **Crescimento Estritamente Linear:** A partir dos 32768 bytes, o tempo de execução começa a subir de forma diretamente proporcional ao tamanho do ficheiro. O algoritmo SHA-256 processa os dados de entrada partindo-os em blocos de tamanho fixo (512 bits). Deste modo, a complexidade temporal é O(N), onde N é o número de blocos do ficheiro. Esta linearidade perfeita confirma que o tempo gasto é ditado exclusivamente pelo volume de dados a ser "digerido"pela função iterativa do *hash*.

5 Análise Comparativa de Desempenho

Nesta secção final, cruzamos os dados obtidos nas alíneas anteriores para responder diretamente às comparações exigidas no guião do projeto.

Importa notar que a comparação entre a **Cifragem e Decifragem RSA** já foi abordada e justificada na Secção 3 deste relatório.

Também decidimos não colocar no relatório os códigos de comparação uma vez que consistem praticamente apenas no desenho do gráfico e não achamos essa parte relevante de colocar. Seja como for estão acessíveis no zip com todos os códigos nos ficheiros `aes_vs_sha256.py` e `aes_vs_rsa.py`.

5.1 Cifragem: AES vs RSA-2048

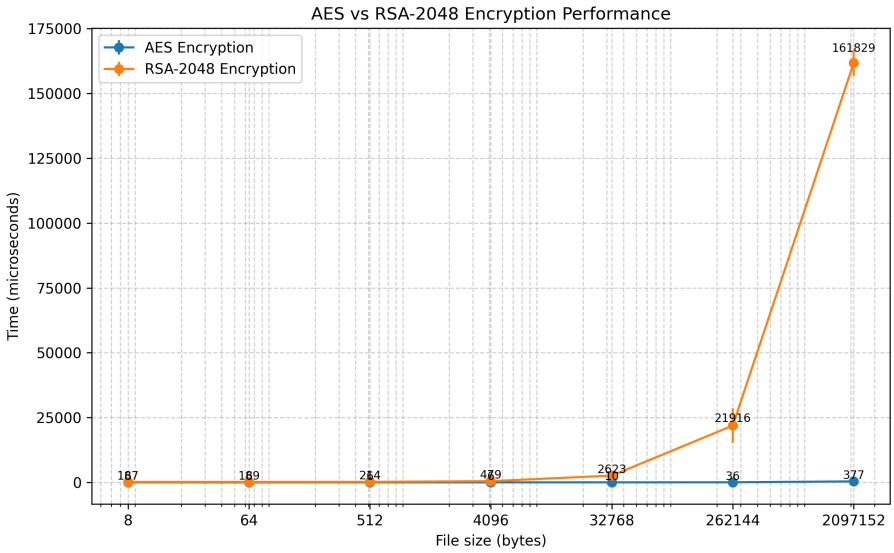

Figura 5: Comparação de desempenho: Cifragem Simétrica (AES) vs Cifragem Assimétrica (RSA).

Da análise deste confronto, retiramos as seguintes conclusões técnicas:

- **Custo Inicial (Overhead):** Para ficheiros pequenos (8 a 4096 bytes) embora não seja muito percetível, o AES demora sempre bastante menos do que o RSA. Esta diferença deve-se ao facto de o AES usar operações simples e aceleradas por hardware, enquanto o RSA envolve aritmética pesada e exponenciação modular com números grandes, resultando num custo inicial muito superior.

- **Escalabilidade e Implementação:** À medida que o ficheiro cresce, ambas as curvas sobem de forma linear. No entanto, a taxa de crescimento (o declive da reta) do esquema RSA é visivelmente superior. Importa clarificar que o nosso esquema RSA cifra os blocos usando *hashes* SHA-256 executados num ciclo `for` nativo em Python. O AES, por sua vez, foi executado através da biblioteca `cryptography`, que delega o trabalho pesado para código otimizado em C.

- **Conclusão Prática:** Os resultados corroboram a teoria criptográfica clássica: algoritmos de chave pública (RSA) são demasiado lentos e computacionalmente pesados para cifrar grandes volumes de dados. Na prática, e tal como o nosso próprio esquema híbrido demonstrou, o RSA deve ser usado apenas para distribuição de chaves ou assinaturas digitais, delegando a encriptação dos dados propriamente ditos para algoritmos simétricos ultrarrápidos como o AES.

5.2 Cifragem AES vs Geração de Digests SHA-256

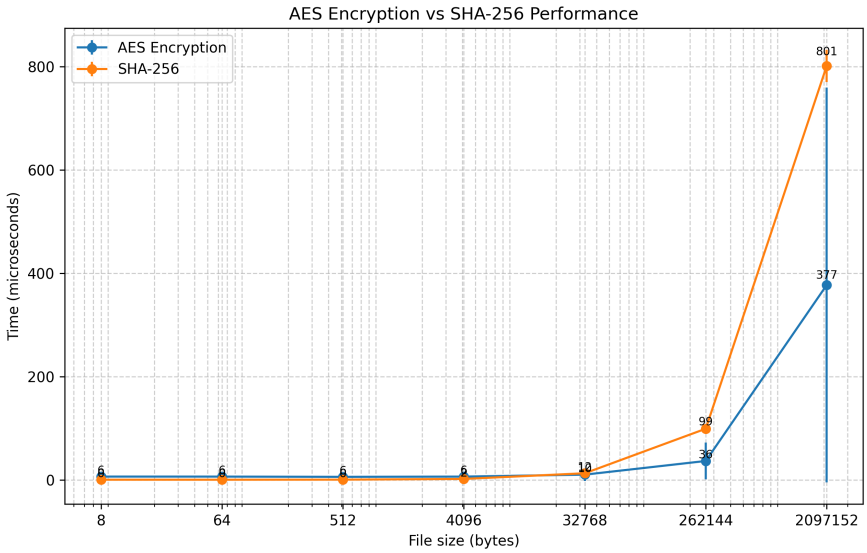

Figura 6: Comparação de desempenho: Cifragem AES vs Geração de Hash SHA-256.

Ao analisar este gráfico, deparamo-nos com um comportamento fascinante e menos intuitivo à primeira vista, que se divide em duas fases distintas:

- **SHA-256 em Ficheiros Pequenos:** Até 4096 bytes, o SHA-256 é mais rápido (0–2 μs) que o AES (6 μs), pois não sofre o *overhead* de inicialização e expansão de chave do AES, processando blocos pequenos quase instantaneamente.
- **AES em Ficheiros Grandes:** A partir de 32768 bytes, o AES supera o SHA-256 (377 vs. 784 μs) graças ao modo CTR paralelizável (vários blocos podem ser processados simultâneamente). SHA-256 permanece sequencial, limitando o desempenho à medida que os dados crescem.

Conclusão

O AES mostrou escalabilidade excecional em ficheiros grandes, com o modo CTR permitindo até superar a velocidade de hashing. O RSA revelou alto custo de aritmética de precisão múltipla (chaves de 2048 bits), tornando-se impraticável para grandes volumes e adequado apenas para proteger pequenas informações, como chaves de sessão. O SHA-256 processou ficheiros pequenos quase instantaneamente devido ao baixo overhead, mas a execução sequencial impediu tirar proveito do paralelismo de hardware, resultando em maior tempo para ficheiros maiores. Assim, sistemas modernos combinam os pontos fortes de cada algoritmo: RSA para troca de chaves, AES para cifrar dados em massa e SHA-256 para garantir integridade.